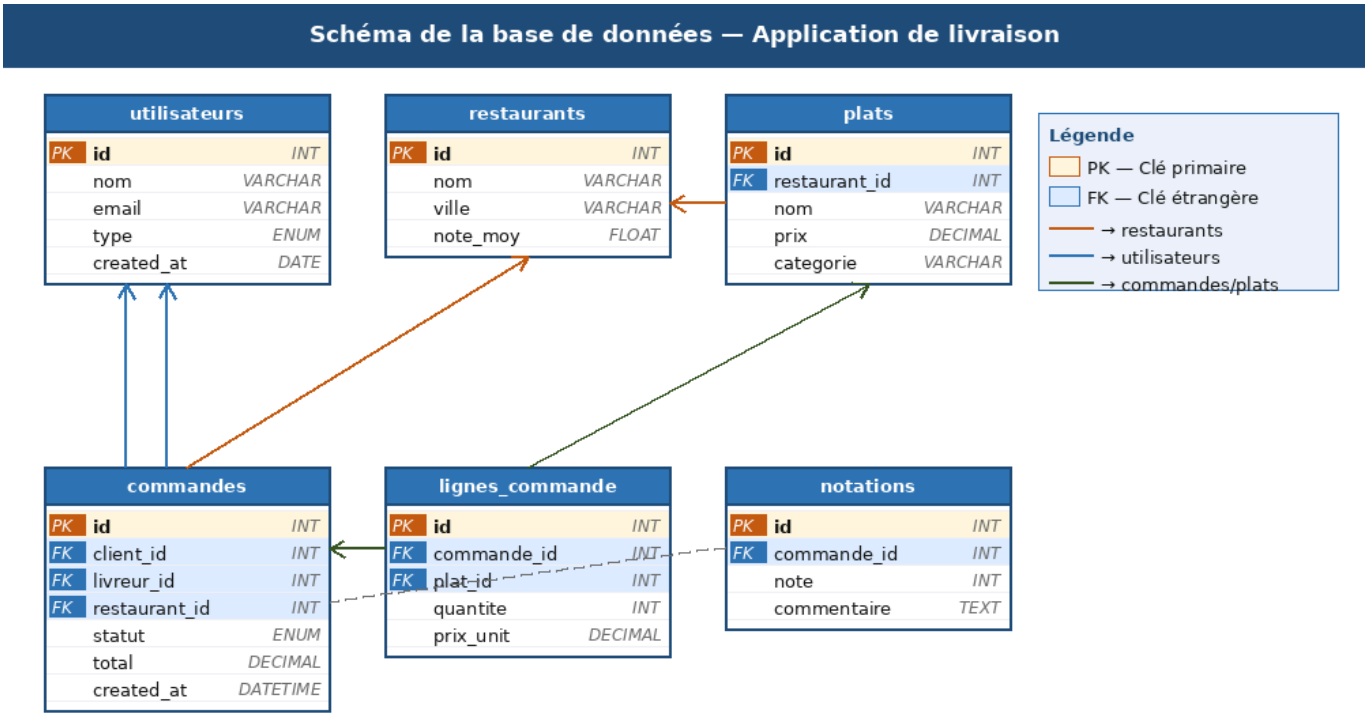
# SQL — Requêtes & Jointures

Semaine 2 · Jeudi · 1h · C5 — Créer une base de données | C6 — Composants d'accès aux données

**Base de données : Application mobile de livraison de repas**

## 1. Schéma de la base de données

Voici la base de données de l'application de livraison que nous avons modélisée hier. Elle contient 6 tables liées par des clés étrangères.




| |
|---|
| PK (orange) = Clé primaire — identifiant unique de chaque ligne |
| FK (bleu) = Clé étrangère — référence vers la PK d'une autre table |
| La table commandes est centrale : elle relie clients, livreurs, restaurants et lignes_commande |

## 2. Requêtes de base

## 2.1 SELECT — Lire des données

### Sélectionner toutes les commandes

```
SELECT * FROM commandes;
```

### Sélectionner des colonnes précises

```
SELECT id, statut, total
FROM commandes;
```

### Filtrer avec WHERE

```
-- Toutes les commandes livrées
SELECT * FROM commandes
WHERE statut = 'livré';

-- Commandes avec un total supérieur à 20€
SELECT * FROM commandes
WHERE total > 20;

-- Commandes livrées ET total > 20€
SELECT * FROM commandes
WHERE statut = 'livré' AND total > 20;
```

### Trier avec ORDER BY et limiter avec LIMIT

```
-- Les 5 commandes les plus récentes
SELECT * FROM commandes
ORDER BY created_at DESC
LIMIT 5;

-- Les plats triés du moins cher au plus cher
```

```
SELECT nom, prix FROM plats
ORDER BY prix ASC;
```

### Recherche partielle avec LIKE

```
-- Restaurants dont le nom contient 'pizza'
SELECT * FROM restaurants
WHERE nom LIKE '%pizza%';

-- Plats dont la catégorie commence par 'burger'
SELECT * FROM plats
WHERE categorie LIKE 'burger%';
```

## 2.2 Fonctions d'agrégation

| Fonction | Utilité — Exemple sur la BDD livraison |
|---|---|
| **COUNT(*)** | Compter le nombre de lignes — ex : combien de commandes au total ? |
| **SUM(col)** | Additionner — ex : chiffre d'affaires total de tous les restaurants |
| **AVG(col)** | Moyenne — ex : note moyenne des livreurs |
| **MIN(col)** | Valeur minimale — ex : commande la moins chère |
| **MAX(col)** | Valeur maximale — ex : commande la plus chère |

```
-- Nombre total de commandes
SELECT COUNT(*) AS nb_commandes FROM commandes;

-- Chiffre d'affaires total
SELECT SUM(total) AS ca_total FROM commandes;

-- Panier moyen
```

```
SELECT AVG(total) AS panier_moyen FROM commandes;
```

### 2.3 GROUP BY & HAVING

GROUP BY regroupe les lignes ayant la même valeur dans une colonne. HAVING filtre les groupes (comme WHERE mais après agrégation).

```
-- Nombre de commandes par statut
SELECT statut, COUNT(*) AS nb
FROM commandes
GROUP BY statut;

-- Chiffre d'affaires par restaurant (seulement ceux > 500€)
SELECT restaurant_id, SUM(total) AS ca
FROM commandes
GROUP BY restaurant_id
HAVING SUM(total) > 500;
```

## 3. Les jointures

Une jointure combine les lignes de plusieurs tables en se basant sur une condition de correspondance. On les utilise dès que les données utiles sont réparties dans plusieurs tables.

### 3.1 INNER JOIN — Seulement les lignes qui correspondent

**INNER JOIN — commandes ⋈ utilisateurs (clients)**

commandes

| id | client_id | statut | tot |
|---|---|---|---|
| 1 | 3 | livré | 24. |
| 2 | 1 | en cours | 18. |
| 3 | 3 | livré | 32. |
| 4 | 5 | annulé | 12. |

JOIN

utilisateurs

| id | nom | type |
|---|---|---|
| 1 | Karim | client |
| 2 | Sara | livreur |
| 3 | Mehdi | client |
| 4 | Youssef | livreur |
| 5 | Fatima | client |

**Résultat INNER JOIN (ON commandes.client_id = utilisateurs.id) :**

| id | client_id | statut | total | nom | type |
|---|---|---|---|---|---|
| 1 | 3 | livré | 24.50 | Mehdi | client |

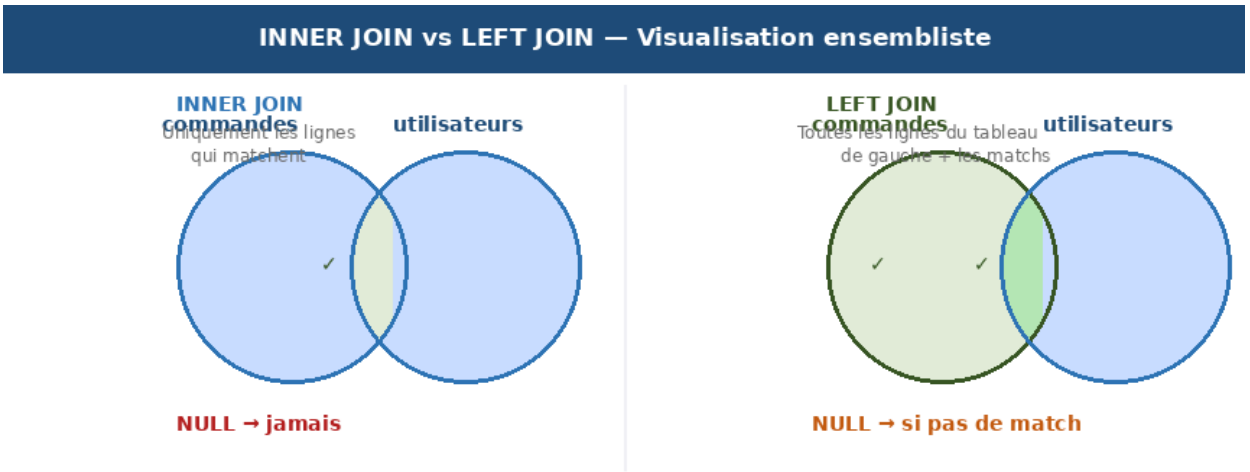
→ lignes communes uniquement

```
-- Commandes avec le nom du client
SELECT c.id, c.statut, c.total, u.nom AS client
FROM commandes c
INNER JOIN utilisateurs u ON c.client_id = u.id;

-- Commandes avec le nom du client ET du restaurant
SELECT c.id, c.statut, c.total,
       u.nom AS client, r.nom AS restaurant
FROM commandes c
INNER JOIN utilisateurs u ON c.client_id = u.id
INNER JOIN restaurants r  ON c.restaurant_id = r.id;
```

Astuce : utiliser des alias (c, u, r) pour raccourcir les noms de tables et éviter les ambiguïtés.

## 3.2 LEFT JOIN — Toutes les lignes de gauche




LEFT JOIN retourne toutes les lignes de la table de gauche, même si aucune correspondance n'existe dans la table de droite (la valeur sera NULL).

```
-- Tous les restaurants, même ceux sans commande
SELECT r.nom, COUNT(c.id) AS nb_commandes
FROM restaurants r
LEFT JOIN commandes c ON c.restaurant_id = r.id
GROUP BY r.id, r.nom;

-- Tous les livreurs, même sans mission ce mois-ci
SELECT u.nom, COUNT(c.id) AS nb_livraisons
FROM utilisateurs u
LEFT JOIN commandes c ON c.livreur_id = u.id
WHERE u.type = 'livreur'
GROUP BY u.id, u.nom;
```

## 3.3 Jointure sur 3 tables

On peut enchaîner plusieurs JOIN dans la même requête pour croiser 3 tables ou plus.

```
-- Détail complet d'une commande :
-- client + restaurant + plats commandés
SELECT u.nom AS client,
```

```
        r.nom AS restaurant,
        p.nom AS plat,
        lc.quantite,
        lc.prix_unit
FROM commandes c
INNER JOIN utilisateurs u  ON c.client_id    = u.id
INNER JOIN restaurants r   ON c.restaurant_id = r.id
INNER JOIN lignes_commande lc ON lc.commande_id = c.id
INNER JOIN plats p         ON lc.plat_id     = p.id
WHERE c.id = 1;
```

## 4. Exercices pratiques

### ✏ Exercice 1 — Requêtes simples — Application de livraison (guidé)

**Contexte :**

En utilisant le schéma de la base de données de l'application de livraison présenté en section 1, écrivez les requêtes SQL suivantes. Testez chaque requête dans MySQL Workbench.

**Requêtes à écrire :**

1. Afficher le nom et l'email de tous les utilisateurs de type 'client'
2. Afficher tous les plats dont le prix est inférieur à 15€, triés du moins cher au plus cher
3. Compter le nombre de commandes par statut ('livré', 'en cours', 'annulé')
4. Afficher les 3 restaurants avec la meilleure note moyenne (ORDER BY + LIMIT)
5. Calculer le chiffre d'affaires total et le panier moyen de toutes les commandes livrées
6. Afficher tous les plats dont le nom contient le mot 'poulet' (LIKE)

### ✏ Exercice 2 — Jointures — Application de livraison (semi-autonome)

**Contexte :**

Ces requêtes nécessitent de croiser plusieurs tables. Utilisez les alias de tables (c, u, r, p, lc) pour rendre les requêtes lisibles. Référez-vous au schéma de la section 1.

**Requêtes à écrire :**

7. Afficher le nom du client et le total pour chaque commande (INNER JOIN commandes + utilisateurs)

8. Afficher le nom du restaurant et le nombre de commandes reçues, même pour les restaurants sans commande (LEFT JOIN)
9. Lister toutes les commandes livrées avec le nom du client, le nom du livreur et le nom du restaurant
10. Afficher les plats commandés au moins une fois avec leur quantité totale vendue (JOIN + GROUP BY + SUM)
11. Trouver les clients qui ont commandé plus de 3 fois, avec leur nombre de commandes (JOIN + GROUP BY + HAVING)
12. BONUS : Afficher le top 3 des livreurs les mieux notés (jointure sur notations + commandes + utilisateurs)

## 5. Récapitulatif des mots-clés SQL

| Clause | Rôle — Ordre d'écriture |
|---|---|
| **SELECT** | Colonnes à afficher → 1er |
| **FROM** | Table source → 2e |
| **JOIN…ON** | Jointure avec une autre table → 3e (autant que nécessaire) |
| **WHERE** | Filtre sur les lignes → 4e |
| **GROUP BY** | Regroupement → 5e |
| **HAVING** | Filtre sur les groupes → 6e |
| **ORDER BY** | Tri du résultat → 7e |
| **LIMIT** | Nombre max de lignes → 8e |

Rappel : WHERE filtre les lignes AVANT le regroupement, HAVING filtre les groupes APRÈS.

INNER JOIN = intersection | LEFT JOIN = toutes les lignes de gauche + correspondances de droite (NULL si absent)